Search Ranking

matching is easy; the best result first is the job

Watch the 4-min video

In one line: engines score text matches with **BM25** (rare terms count more, repeated terms saturate, long documents are normalised), then shape the score with **field boosts** and **business signals**, optionally re-rank the top results with a **learning-to-rank** model, and prove every change with offline metrics (NDCG, MRR) and A/B tests.

1 Same products, different results

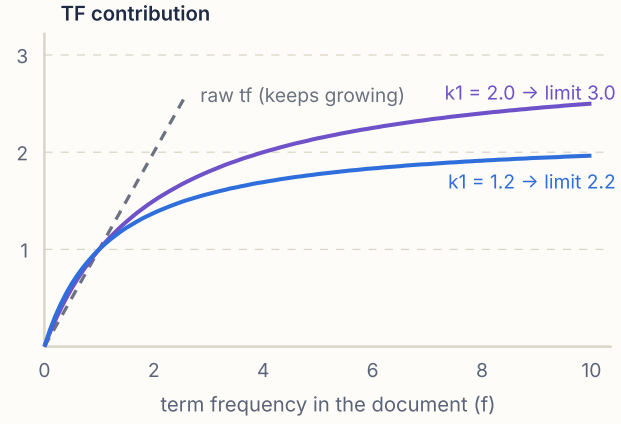
Two shops sell the same catalogue on the same engine. Typing **wireless headphones** in one shows the best-selling in-stock pair first; the other shows a charging cable and a sold-out item. Both found the right documents. Ranking decides whether the user sees them: most clicks go to the first few results, so positions 1–5 are what relevance work is about.

2 TF-IDF, then BM25

TF-IDF multiplies **term frequency** (a document that says 'headphones' often is probably about headphones) by **inverse document frequency** (a term that appears in few documents is more informative). Raw TF has two flaws: the score grows without limit, so keyword stuffing wins; and long documents mention everything, so they win too. **BM25** (Okapi BM25, the default similarity in Lucene since Elasticsearch 5.0) fixes both.

$$\text{score} = \Sigma_t \ \text{IDF}(t) \cdot f(k1+1) \ / \ (f + k1(1 - b + b \cdot |D|/\text{avgdl}))$$

Here f is how often term t appears in document D, |D| is the field length and avgdl the average length of that field. Lucene uses IDF = ln(1 + (N − n + 0.5)/(n + 0.5)), where n of N documents contain the term. Scores are computed **per field**, and the statistics are per shard by default, so tiny indices with uneven shards can rank slightly inconsistently (`search_type=dfs_query_then_fetch` gathers global statistics first, at the cost of an extra round trip).



Length normalisation (b)

k1 is multiplied by (1 − b + b · |D| / avgdl)
|D| short (title) → smaller → higher score
|D| long (manual) → bigger → lower score
b = 0: ignore length; b = 1: full effect
defaults: k1 = 1.2, b = 0.75

IDF: rare terms weigh more

N = 1 M docs: 'the' in 900 k → IDF ≈ 0.1
'wireless' in 5 k → IDF ≈ 5.3
'headphones' in 2 k → IDF ≈ 6.2

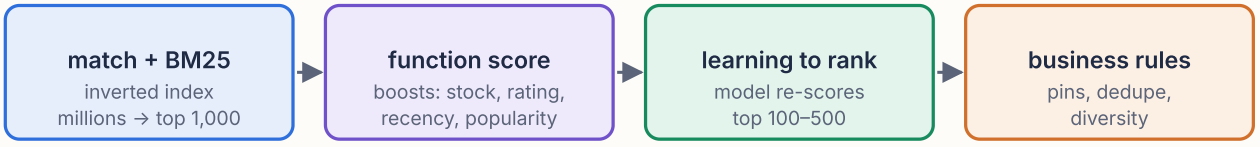
Figure 1. Left: with BM25 the 10th occurrence adds little; the curve flattens toward k1 + 1. Right: what b and IDF do.

Occurrences f	1	2	3	5	10	limit
BM25 TF part (k1 = 1.2, average length)	1.00	1.38	1.57	1.77	1.96	2.20
raw TF	1	2	3	5	10	∞

3 Shaping the score

- **Field boosts**: a match in `title^3` counts more than one in `description`. `multi_match` types matter: `best_fields` takes the best single field, `most_fields` adds them up, `cross_fields` treats fields as one big field (good for names split into first/last).
- **Phrase and exact boosts**: add an optional `match_phrase` clause so documents with the words together rank higher.
- **Function score**: multiply or add business signals. `field_value_factor` on popularity or rating with a `log1p` / `log2p` modifier (note: these are $\log_{10}$ in Elasticsearch) so 10,000 sales doesn't beat 100 by 100×; **decay functions** (`gauss`, `exp`, `linear`) for recency or distance; filter-based `weight`s for flags like in-stock.
- **Combine with care**: `boost_mode: multiply` keeps relevance in charge; `sum` or `replace` can let popularity override what the user typed.

4 Learning to rank: two-stage ranking



cheap → expensive per document; each stage sees fewer documents

Elasticsearch: query + function_score, then a rescore (window_size) or the LTR rescorer on the top hits

Figure 2. A ranking funnel. Expensive scoring only runs on the few hundred candidates that survive the cheap stage.

For big catalogues, hand-tuned boosts plateau. **Learning to rank (LTR)** trains a model, typically gradient-boosted trees such as LambdaMART (XGBoost, LightGBM), on judgement lists built from clicks, add-to-carts and purchases, or from human graders. Features include BM25 per field, phrase match, price, rating, sales, freshness, and user or query context. Because the model is slow compared with BM25, it only **re-scores the top N** (a few hundred) from the first stage. Recent Elasticsearch 8.x versions have a built-in LTR rescorer; OpenSearch and older Elasticsearch use the LTR plugin. Training on clicks needs care: users click higher positions anyway (**position bias**), so raw click-through over-rewards what you already rank first.

5 Measuring relevance

Build a **judgement set**: a few hundred real queries (head and long tail), each with results graded e.g. 0–3. Then compute:

- **NDCG@k**: DCG@k = $\Sigma\ (2^{rel_i} - 1) / \log_2(i + 1)$ over the top k positions, divided by the DCG of the ideal ordering, so 1.0 is perfect. It rewards relevant results near the top and handles graded relevance.
- **MRR**: the average of 1/rank of the first relevant result. First good result at position 1 → 1.0; at position 4 → 0.25. Ideal for 'find one thing' queries such as navigation.
- **Precision@k / recall@k**: share of the top k that is relevant / share of all relevant docs that made the top k.

- **Online**: click-through rate, zero-result rate, search-to-purchase conversion, time to first click. Compare rankings with **A/B tests** or the faster **interleaving** (mix both rankings in one list and see which side's results get clicked).

Example NDCG@3: results graded [2, 0, 1] give DCG = 3/1 + 0/1.58 + 1/2 = 3.5; the ideal order [2, 1, 0] gives 3 + 1/1.58 + 0 = 3.63; NDCG@3 = 3.5 / 3.63 ≈ **0.96**. Swapping the irrelevant item out of position 2 is worth 0.04.

WORKED EXAMPLE: BOOSTING IN-STOCK PRODUCTS

Query **wireless headphones**. A `function_score` wraps the BM25 query with three functions, combined by `score_mode: multiply` and applied with `boost_mode: multiply`: weight **2** if `in_stock: true`, weight **0.5** if false, and `field_value_factor(rating, modifier: log2p)` = $\log_{10}(2 + \text{rating})$. (The video says log1p; log1p of a 0 rating is 0, which would wipe out an unrated product's score in multiply mode, so log2p or `missing` is safer.)

Product	BM25	stock	rating factor	final
Sony wireless headphones	9.0	×2	4.7 → 0.83	**14.9**
Wireless headphones charging cable	9.5	×2	3.2 → 0.72	13.6
Bose wireless headphones (sold out)	9.8	×0.5	4.8 → 0.83	4.1

The available, well-rated headphones now lead; the sold-out pair is still findable, just lower. The cable still ranks second because its text matches well: a category boost ('headphones' query → Headphones category) or an LTR model fixes that. The change ships only after the A/B test shows higher click-through and conversion.

TRADE-OFFS

- **Business boosts** lift revenue but can bury the most relevant item: watch zero-click and refinement rates.
- **LTR** beats hand-tuning at scale but needs judgement data, feature logging and model ops.
- **Higher k1** rewards repetition more; **lower b** stops penalising long documents (good for short, uniform fields).
- **Global statistics (DFS)** give consistent scores across shards at the cost of an extra round trip.

COMMON MISTAKES

- Tuning relevance by eyeballing a few favourite queries instead of a judgement set.
- Adding popularity with `boost_mode: sum`, so bestsellers appear for unrelated queries.
- Training on raw clicks without correcting for position bias.
- Comparing BM25 scores across different queries or indices: they are only meaningful within one result list.

6 Key terms

IDF: inverse document frequency: rare terms weigh more

BM25: TF-IDF with TF saturation (k1) and length normalisation (b)

Function score: mixes business signals into the text score

Learning to rank: ML model re-scores the top candidates

NDCG: graded, position-discounted ranking quality, 0–1 **MRR**: mean of 1/rank of the first relevant result

7 Interview questions

Q1. How does BM25 score documents? BASIC

For each query term it combines saturating term frequency, inverse document frequency (rare terms count more) and document-length normalisation, then sums across terms.

Q2. How do you measure relevance? BASIC

Offline with judged queries and metrics like NDCG and MRR; online with click-through, conversions, zero-result rate and A/B tests or interleaving.

Q3. What do k1 and b control? BASIC

k1 sets how quickly term frequency saturates (default 1.2); b sets how strongly long fields are penalised, from 0 (none) to 1 (full), default 0.75.

Q4. Sold-out items rank first. Fix it without hiding them. SCENARIO

Wrap the query in function_score with a weight > 1 for in-stock and < 1 for out-of-stock, multiply mode, and verify with an A/B test.

Q5. You have millions of products and good click logs. How do you improve ranking further? SCENARIO

Two-stage ranking: BM25 plus boosts retrieves the top few hundred, an LTR model trained on debiased clicks and purchases re-scores them; measure with NDCG offline and A/B online.

Q6. A query returns zero results 8% of the time. What do you look at? SCENARIO

Log and cluster the zero-result queries: missing synonyms, typos (fuzziness), over-strict AND matching (use minimum_should_match), or analyzer gaps; then fall back to a looser query.

Q7. Why might a product rank differently on two identical queries? TRICKY

Requests can hit different shard copies, and BM25 statistics are per shard and include not-yet-merged deleted docs; use a preference key or DFS for consistency.

Q8. The team says the new ranking 'feels better' but the A/B test is flat or negative. Ship it? TRICKY

No: user behaviour is the ground truth; check segment effects and the judgement set, but don't ship on intuition.

Next: SD-P-005 · Filtering and Faceting backendroadmap.com